No-Opponent-Cycle Propagators for Solving Parity Games Appendix

Gonzalo Hernandez[1,3], Julian Garcia[1], Julian Gutierrez[2], and Guido Tack[1]

[1] Monash University, Australia
{gonzalo.hernandez,julian.garcia,guido.tack}@monash.edu
[2] University of Sussex, United Kingdom {J.Gutierrez}@sussex.ac.uk
[3] Universidad de Nariño, Colombia {gonzalo.hernandez}@udenar.edu.co

Appendix A: Additional experiments

This experiment focuses on CP and SMT solvers. We have selected Jurdziński graphs for this experiment, which allows us to vary the size of the graphs. We selected graphs with 10 blocks and between 10 and 80 levels (in steps of 10).

Table 1: Solving time on the Jurdziński Graphs $J_{level,10}$ CP-CP. (*) Represent times exceeding 5 minutes.

d	**No-Opponent-Cycle**		**Other Solvers**		
	□ Eager	○ Memo	Mzn(Gecode)	Mzn(CP-SAT)	Z3
10	0.000	0.008	0.59	0.71	0.22
20	0.000	0.025	3.06	2.59	0.58
30	0.000	0.039	10.57	6.15	2.64
40	0.000	0.087	28.53	12.52	7.76
50	0.001	0.089	63.77	22.47	18.11
60	0.001	0.097	126.88	37.41	35.76
70	0.001	0.158	231.51	58.22	65.62
80	0.001	0.185	*	88.89	111.80

Table 1 presents the solving time in seconds of four approaches: No-Opponent-Cycle, a MiniZinc CP model (expressing the progress measure using integer variables and constraints) solved by Gecode and Google CP-SAT, and an SMT model using difference logic to implement the progress measure, solved using Z3. The results provide strong evidence that Chuffed with the No-Opponent-Cycle propagator clearly outperforms the other approaches by several orders of magnitude on these graphs. Furthermore, some solutions, such as MiniZinc with Gecode, used over 256 GB of memory to complete the task.

In Table 2, we evaluated performance in three classes of arenas: RandGames (RaG), SteadyGames (StG), and ClickGames (ClG). For each class, we generated instances and solved them using the ZRA solver, NOC-Eager (differing by the player of interest), and three SAT-based solvers (CaDiCaL, MiniSat, and Kissat).

Table 2: Average solving times (in seconds) on special game classes, categorized by winner.

Game class	**Win**	**ZRA**	**NOC from ○**		**NOC from □**		**SAT Solvers**		
			Eager	Memo	Eager	Memo	CaDiCal	MiniSat	Kissat
RaG	○	0.009	0.040	0.800	0.003	0.004	10.393	16.069	14.936
RaG	□	0.008	0.004	0.004	0.035	0.776	9.216	6.189	13.369
StG	○	0.009	0.032	0.680	0.003	0.003	132.217	233.167	158.103
StG	□	0.009	0.002	0.003	0.028	0.627	62.470	29.582	32.775
ClG	○	0.010	0.142	1.725	0.001	0.002	20.548	57.217	22.362
ClG	□	0.026	0.005	0.006	0.472	5.632	22.856	8.702	11.572

Table 2 summarizes the average solving times over 35 runs per configuration. Each run starts from a randomly chosen initial vertex v_0. Results are grouped by game class, with the second column indicating the winning player (○ or □) for each configuration. The remaining columns report the average solving times (in seconds) for each solver and variant.

In all instances, every variant of No-Opponent-Cycle outperforms the SAT solvers by at least three orders of magnitude. Moreover, one of the No-Opponent-Cycle variants achieves performance comparable to ZRA, which is the best performing non-CP-based procedure in practice [1]. Notably, when we select p as the player of interest, No-Opponent-Cycle tends to be faster in cases where $\overline{\mathsf{p}}$ wins. This is because the algorithm can often conclude early by detecting the unsatisfiability of the problem.

Table 3: Average solving times (in seconds) on the RandomGamesSet-500, grouped by game size and winner.

Game Size $\lvert V\rvert \times \lvert E\rvert$	**Win**	**ZRA**	**NOC from ○**			**NOC from □**		
			Checker	Eager	Memo	Checker	Eager	Memo
8K × 20	○	0.020	26.041	0.125	2.759	60.000	0.009	0.011
8K × 20	□	0.020	60.000	0.011	0.013	36.396	0.186	3.532
20K × 20	○	0.061	31.347	0.502	14.493	60.000	0.026	0.029
20K × 20	□	0.062	60.000	0.032	0.037	27.250	0.485	14.268
50K × 20	○	0.209	23.382	2.323	60.000	26.437	0.076	0.087
50K × 20	□	0.205	23.085	0.084	0.096	21.387	2.422	60.000
90K × 20	○	0.427	60.000	6.905	60.000	60.000	0.148	0.163
90K × 20	□	0.422	60.000	0.172	0.196	60.000	7.129	60.000
150K × 10	○	0.331	60.000	6.050	60.000	60.000	0.138	0.147
150K × 10	□	0.317	60.000	0.244	0.254	60.000	13.221	60.000

In Table 3 we compared the performance of No-Opponent-Cycle against ZRA using a benchmark of 500 randomly generated games of varying sizes and edge densities. The sizes range from 8,000 to 150,000 vertices, with a maximum of 10 or 20 outgoing edges per vertex. In each game, the initial vertex v_0 was also selected randomly Table 3 reports the average solving times (in seconds), grouped by game size and by the winning player (○ or □). Each row presents the performance of ZRA, along with two No-Opponent-Cycle variants from Player ○'s perspective (Eager, Memo) and two corresponding variants from Player □'s perspective. The results show that the NOC-Eager variant was consistently competitive with ZRA, often achieving similar or better solving times. To support these observations, we conducted a standard deviation analysis and a Wilcoxon signed-rank test, both of which provide strong statistical evidence. These results, along with the full version of the table, are provided in the appendix, which also highlights the differences between the No-Opponent-Cycle filters and the Checker variant; a timeout of 60 seconds was set on each solver run.

Table 4: Solving times on Jurdziński graphs $J_{d,10}$. Symbol (*) represents unsolvable by the solver. [3].

d	**ZRA**	**No-Opponent-Cycle**		**SAT Solvers**			
		□ *Eager*	○ Memo	CaDiCaL	MiniSat	Kissat	zChaff
10	0.009	*0.000*	0.008	0.130	0.292	1.040	0.156
30	0.140	*0.000*	0.039	1.700	3.073	8.670	0.670
50	0.564	*0.001*	0.089	5.680	9.329	15.550	1.558
70	1.451	*0.001*	0.158	12.940	19.726	26.320	2.930
90	2.975	*0.001*	0.240	93.150	34.565	41.140	*
110	5.309	*0.001*	0.344	134.840	55.368	62.930	
130	8.616	*0.001*	0.468	193.500	80.627	85.320	*
150	13.070	*0.002*	0.610	260.590	111.662	115.420	
170	18.826	*0.002*	0.769	342.000	148.768	147.460	*
190	26.131	*0.002*	0.956	437.110	192.320	182.210	
210	35.045	*0.002*	1.145	539.770	244.924	230.440	*
230	45.780	*0.002*	1.363	655.450	297.770	283.340	
250	58.552	*0.002*	1.599	749.670	368.385	341.040	*
270	73.619	*0.003*	1.866	879.440	447.689	395.860	
290	90.890	*0.003*	2.127	1057.340	539.538	468.190	*
310	110.554	*0.003*	2.420	1207.110	623.698	540.090	
330	132.978	*0.003*	2.819	1403.980	717.835	585.110	*
350	158.433	*0.003*	3.197	1588.480	809.985	630.480	
370	187.527	*0.004*	3.562	1767.350	928.425	711.390	*
390	219.209	*0.004*	3.941	2113.110	1039.260	818.460	
410	254.540	*0.004*	4.348	2341.200	1157.990	897.660	*
430	292.778	*0.004*	4.742	2612.670	1387.010	991.170	

The Table 4 shows the set of experiments compares the No-Opponent-Cycle propagator with ZRA and SAT-based approaches, using three different solvers: CaDiCaL, MiniSat, and Kissat. For completeness, we included results for zChaff,

as it was used in previous work [2], though it is no longer competitive. Table 4 shows solving times in seconds. In this experiment, we extended the range of levels (and thus the problem size) up to 410. The NOC-Memo propagator, when solving from player ○ 's perspective, consistently outperforms SAT-based approaches by at least two orders of magnitude and exceeds ZRA's performance by more than one order of magnitude. Moreover, the full version of the table, provided in the appendix, shows that when the problem is flipped to search from player □ 's perspective, NOC-Eager performs even faster, achieving speedups of over four orders of magnitude compared to ZRA on the largest instances.

All the results reported above omit initialization times. For example, SAT solvers required over 300 seconds to initialize on the largest instances, whereas No-Opponent-Cycle took less than 30 milliseconds in the same game. In contrast, ZRA does not involve a significant initialization phase.

Appendix B: Statistical Evaluation

Table 5: Statistical analysis based on Table 3 comparing ZRA and NOC-Eager from □'s perspective. The table includes mean (AVG) and standard deviation (SD) of solving times, along with Wilcoxon signed-rank test statistics (Samples, WR+, WR-).

Game Size $\lvert V\rvert \times \lvert E\rvert$	**Win**	**ZRA** AVG	SD	**NOC from □** AVG	SD	**Wilcoxon SR** Samples	WR+	WR-
8K × 20	○	0.020	0.001	0.009	0.001	84	0	-3570
20K × 20	○	0.061	0.061	0.026	0.002	55	0	-1540
50K × 20	○	0.209	0.025	0.076	0.006	44	0	-990
90K × 20	○	0.427	0.094	0.148	0.019	50	0	-1275
150K × 10	○	0.331	0.082	0.138	0.006	24	0	-520

Table 6: Statistical analysis based on Table 3 comparing ZRA and NOC-Eager from ○'s perspective. The table includes mean (AVG) and standard deviation (SD) of solving times, along with Wilcoxon signed-rank test statistics (Samples, WR+, WR-).

Game Size $\lvert V\rvert \times \lvert E\rvert$	**Win**	**ZRA** AVG	SD	**NOC from ○** AVG	SD	**Wilcoxon SR** Samples	WR+	WR-
8K × 20	□	0.020	0.001	0.011	0.004	74	75	-2700
20K × 20	□	0.062	0.004	0.032	0.014	45	10	-1025
50K × 20	□	0.205	0.028	0.084	0.018	56	0	-1596
90K × 20	□	0.422	0.100	0.172	0.052	46	1	-1080
150K × 10	□	0.317	0.040	0.244	0.148	22	33	-220

SOTA methods (ZRA) cannot exploit this duality; as shown in Tables 1–2, their performance remains consistent regardless of which player wins (we have also verified this for all original games and their complements)

The statistical evidence presented in Table 5 and Table 6, suggests that NOC-Eager achieves comparable performance to ZRA, with a consistent trend in its favor. In all cases, NOC-Eager records lower average solving times with smaller standard deviations, indicating slightly more stable behavior. The Wilcoxon signed-rank test supports this observation: the number of positive ranks (WR+) is generally low, and the negative rank sums (WR–) are consistently large, providing statistically significant—though not drastic—evidence that NOC-Eager tends to perform better across the tested instances.

References

1. Aggarwal, S., de la Banda, A.S., Yang, L., Gutierrez, J.: A matrix-based approach to parity games. In: Sankaranarayanan, S., Sharygina, N. (eds.) Tools and Algorithms for the Construction and Analysis of Systems. pp. 666–683. Springer Nature Switzerland, Cham (2023)
2. Heljanko, K., Keinänen, M., Lange, M., Niemelä, I.: Solving parity games by a reduction to SAT. Journal of Computer and System Sciences **78**(2), 430–440 (2012). https://doi.org/10.1016/j.jcss.2011.05.004, games in Verification
3. Jurdziński, M.: Small progress measures for solving parity games. In: Reichel, H., Tison, S. (eds.) STACS 2000. pp. 290–301. Springer Berlin Heidelberg, Berlin, Heidelberg (2000)